

PhishGuard — Project Report

FutureTech HackFest 2026 · Theme: *Innovating Tomorrow with Emerging Technologies* Focus areas: **Applied AI · Cybersecurity** Author: Chiara De Venuto (individual participation)

1. Executive Summary

PhishGuard is an offline-first AI email phishing detector. It combines a **machine learning classifier** (TF-IDF text representation + logistic regression) with **15 interpretable heuristic features** to produce a 0-100 phishing risk score, a verdict (SAFE / SUSPICIOUS / PHISHING) and — crucially — **evidence**: the exact features and text signals that drove the prediction. Unlike black-box commercial filters, PhishGuard explains *why* an email is dangerous, which is what makes it usable for education, awareness campaigns and small organizations without cloud dependencies.

Key results: 98.61% holdout accuracy, 98.89% 5-fold cross-validation accuracy, 0 false positives, 2 false negatives on a balanced 720-email curated dataset.

2. Problem

Phishing remains one of the most effective initial-access vectors in cybersecurity. APWG consistently reports record phishing volumes: millions of unique phishing sites per quarter, with social engineering increasingly targeting individuals rather than infrastructure. For an individual user or a small organization, the question is rarely "is this link malicious?" (blacklists answer that) but "**is this email trustworthy?**" — which is exactly where detection must operate *before* the click.

Two practical gaps motivated this project: 1. **Explainability** — most detectors return a verdict without reasons; users do not learn and cannot defend themselves elsewhere. 2. **Dependency** — commercial solutions are cloud-based; an email sent to a device with no connectivity, or in a privacy-sensitive context, cannot be analyzed. PhishGuard runs 100% locally.

3. Proposed Solution

email text → TF-IDF (1-2 grams, 1671 features) → hstack → Logistic Regression
→ score 0-100 → 15 heuristic features (scaled) → | verdict + evidence ←

- **Web UI** (Flask): paste an email, get score bar, verdict badge, advice, active heuristic features and top contributing text indicators.
- **JSON API** (POST /api/analyze): programmatic use, integration-ready.

Why this approach

- **No external API keys, no cloud, no data exfiltration** — the model is a ~300 KB local artifact. A privacy-first design.
- **Hybrid signals** — TF-IDF captures subtle textual patterns; heuristics encode expert knowledge (typosquatting TLDs, raw-IP URLs, Reply-To mismatches, urgency/financial trigger words) that ML on small data may miss.
- **Interpretability by construction** — logistic regression coefficients are directly readable; the UI surfaces them.

4. Methodology

4.1 Dataset

Balanced, labeled synthetic corpus of **720 emails (360 phishing / 360 legit)**, generated deterministically (seed 42) from documented phishing patterns: typosquatted domains, raw-IP URLs, urgent/financial trigger phrases, generic greetings, Reply-To mismatches, malicious attachments, ALL-CAPS manipulation — *and* **hard examples** that make the task non-trivial:

- legitimate emails containing phishing-like words ("verify your email", "password changed", "security settings updated") with legitimate domains;
- low-intensity phishing (calm tone, no exclamation marks).

No real user data was used. The generator is part of the repository (`phishguard/build_dataset.py`), making the dataset fully reproducible.

4.2 Features

Text: TF-IDF, n-grams 1-2, 4000 max features (1671 after `min_df=2`), sublinear TF.

Heuristics (15): `num_urls`, `has_ip_url`, `n_click_here`, `n_suspicious_tld`, `urgent_keywords`, `financial_keywords`, `generic_greeting`, `n_exclamations`, `n_all_caps_words`, `caps_ratio`, `body_length`, `n_attachments`, `has_attachment_word`, `reply_to_mismatch`, `has_phone`.

4.3 Model

Logistic Regression (`C=1.0`, `class_weight=balanced`, 3000 iterations) on the stacked sparse matrix. Logistic regression was chosen deliberately: strong baseline for text classification, fast to train/evaluate on modest hardware, and **fully interpretable**.

4.4 Evaluation

- Holdout: stratified 80/20, seed 42 → **98.61% acc, 100% precision, 97.22% recall, F1 98.59%**. Confusion matrix: `[[72, 0], [2, 70]]` — 0 false positives, 2 false negatives.
- **5-fold CV** (mean): 98.89% acc, F1 98.87%.
- **Ablation:** text-only and heuristics-only both reach ~99% CV accuracy on this corpus; the combined pipeline is the production choice and the heuristics supply the evidence layer for the UI.

5. Innovation & Uniqueness

1. **Explainability as a feature, not an add-on:** every verdict is delivered with the list of active heuristic signals and the top text contributors. The user learns *what to look for* — directly supporting security awareness, a recognized first line of defense.
2. **Offline-first, privacy-preserving detection:** no email content ever leaves the device. Unusual among modern detectors.
3. **Reproducible synthetic corpus:** the entire dataset is generated by committed code — a transparent, auditable training pipeline.
4. **Dual-layer defense:** ML + expert heuristics, with both layers exposed.

6. Limitations & Future Work

- The corpus is synthetic; generalization to real-world mail (Enron corpora, live phishing feeds) must be validated. Public labeled datasets and manual triage are the immediate next step.
- Single-token text indicators are crude explanations; a follow-up will generate natural-language

- rationales (offline LLM or template-based).
- Logistic regression is a strong baseline, not a frontier model; a fine-tuned transformer could improve recall on adversarial examples, at the cost of size and interpretability.
- Roadmap: browser extension, MCP/server integration, threat-intel feed (blacklist) as a third layer, explainability for the SUSPICIOUS band.

7. Tech Stack

Layer	Technology
ML	scikit-learn 1.9 (TF-IDF, LogisticRegression), joblib, scipy
Web	Python 3.12, Flask 3.1
Tests	stdlib unittest (6 end-to-end tests)
Assets	pip requirements only — no external services

8. How to Run

```
bash python3 -m venv .venv .venv/bin/pip install -r requirements.txt .venv/bin/python -m phishguard.build_dataset # optional: rebuild dataset .venv/bin/python -m phishguard.model # optional: retrain .venv/bin/python -m unittest tests.test_app # tests .venv/bin/python app.py # http://127.0.0.1:5001
```

9. AI Tools Disclosure (transparency compliance)

This project was developed with the assistance of AI tools: a local AI coding assistant (APEX) was used for scaffolding, code generation support, debugging and documentation drafting. The code was reviewed, executed and tested locally; all metrics reported in this document were produced by running the code, not by the assistant. No proprietary or personal data was shared with external services.

This disclosure is made in compliance with: - **Regulation (EU) 2024/1689 (EU AI Act), Article 50** (transparency obligations, applicable since 2 August 2026); - **Italian Law No. 132/2025** on artificial intelligence (transparency principles, Article 13).

The purpose of this disclosure is full transparency towards evaluators, users and any third party reviewing this submission.

10. Declaration

This project is original work, developed for the FutureTech HackFest 2026 submission. It uses only open-source libraries. The dataset is generated by code committed to the repository.